

Report

Title

Evaluating Open-Source LLMs for UI Element Selection: Query Ambiguity and Interface Complexity in Developer Tools

Abstract

Large language models (LLMs) are increasingly integrated into intelligent assistants, software agents, and developer-support systems for navigating complex interfaces. However, their ability to correctly identify user interface (UI) elements under varying query ambiguity and interface complexity remains insufficiently explored in developer-oriented environments. This study presents a controlled empirical evaluation of five open-source LLMs — TinyLlama (1.1B), Gemma 2 (2B), Phi-3 Mini (3.8B), Llama 3 (8B), and Mistral 7B — on a structured dataset of 360 query instances across GitHub and AWS interfaces. The dataset follows a balanced 3×2 factorial design involving three ambiguity levels (clear, paraphrased, ambiguous) and two interface sizes (small: 10 elements; large: 21–24 elements).

Results show that Mistral 7B achieves the highest overall accuracy of 90.00%, while TinyLlama performs near random baseline levels (7.22%), indicating a minimum model capability threshold for UI selection tasks. Accuracy decreases consistently as query ambiguity increases, and larger interfaces further amplify performance degradation. Statistical analysis using McNemar’s test reveals that most model differences are significant ($p < 0.05$), except between Gemma 2 (2B) and Llama 3 (8B), suggesting that instruction tuning quality may outweigh parameter count for structured UI selection tasks.

Keywords: large language models, UI element selection, query ambiguity, interface complexity, developer tools, GitHub, AWS, LLM evaluation

1. Introduction

Modern developer platforms such as GitHub and Amazon Web Services contain highly dense interfaces with numerous interactive components. As LLM-powered assistants become integrated into coding environments, automation agents, and task-completion systems, selecting the correct UI element based on user intent becomes increasingly important.

This task is influenced by two major variables:

1. **Query Ambiguity** — differences in how users express the same intent.
2. **Interface Complexity** — the number of candidate UI elements available.

While previous research has evaluated LLM reasoning, dialogue understanding, and web-agent performance, limited work systematically investigates how ambiguity and interface size jointly affect UI element selection accuracy in developer environments.

This study addresses this gap through a controlled experimental framework.

2. Research Gap

Existing research demonstrates strong LLM performance on language understanding and autonomous web navigation tasks. However, several limitations remain:

Gap 1: Lack of Controlled UI Selection Evaluation

Most benchmarks such as GLUE, SuperGLUE, BIG-Bench, and MMLU focus on reasoning and language understanding rather than structured UI selection.

Gap 2: Missing Ambiguity-Level Analysis

Previous work rarely isolates the effect of query ambiguity on model performance.

Gap 3: Limited Study of Interface Complexity

Research generally evaluates full web tasks but does not measure how increasing UI distractors impacts model accuracy.

Gap 4: Lack of Statistical Significance Testing

Many LLM comparisons rely only on raw accuracy without verifying whether differences are statistically meaningful.

Gap 5: Limited Developer Tool Evaluation

Few studies examine LLM interaction with real-world developer environments such as GitHub and AWS.

3. Research Contributions

This work contributes the following:

1. A balanced 360-row benchmark dataset for UI element selection.
 2. Controlled manipulation of ambiguity and interface size.
 3. Evaluation across five open-source LLMs.
 4. Statistical significance testing using McNemar’s test.
 5. Empirical evidence regarding model-size thresholds.
 6. Insights into instruction-tuning quality versus parameter count.
-

4. Dataset Design

4.1 Dataset Overview

The dataset contains 360 instances generated from 60 unique tasks.

- GitHub tasks: 30
- AWS tasks: 30
- Total rows: 360

Each task appears under:

- 3 ambiguity levels
- 2 interface conditions

Thus:

[60 × 3 × 2 = 360]

4.2 Factorial Design

Variable	Levels
Query Ambiguity	Clear, Paraphrased, Ambiguous
Interface Complexity	Small, Large

4.3 Ambiguity Types

Clear Query

Direct mapping to intended action.

Example:

“Create a new repository”

Paraphrased Query

Alternative wording with preserved intent.

Example:

“Start a new project repo”

Ambiguous Query

Implicit wording introducing uncertainty.

Example:

“Set up something new for my code”

4.4 Interface Complexity

Small Interface

- 10 UI options
- 1 correct + 9 distractors

Large Interface

- 21–24 UI options
- 1 correct + 20–23 distractors

Random baseline:

- Small $\approx 10\%$
 - Large $\approx 4.5\%$
-

5. Methodology

5.1 Models Evaluated

Model	Parameters
TinyLlama	1.1B
Gemma 2	2B
Phi-3 Mini	3.8B
Llama 3	8B
Mistral 7B	7B

5.2 Prompt Design

All models received:

- Zero-shot prompt
- Fixed deterministic configuration
- Temperature = 0
- Same UI candidate list

Models were instructed to output only one UI element.

5.3 Evaluation Metrics

Primary Metric:

- Accuracy

Secondary Metrics:

- Weighted F1-score
- Precision
- Recall

Statistical Testing:

- McNemar's paired significance test
-

6. Experimental Results

6.1 Overall Accuracy

Model	Accuracy
TinyLlama	7.22%
Phi-3 Mini	69.72%
Gemma 2	81.67%
Llama 3	82.22%
Mistral 7B	90.00%

6.2 Key Findings

Finding 1: Mistral 7B Performs Best

Mistral 7B achieved the strongest performance across all evaluation settings.

Finding 2: TinyLlama Performs Near Random

TinyLlama demonstrates insufficient capability for structured UI selection.

Finding 3: Ambiguity Is the Strongest Failure Driver

Performance consistently declines from clear → paraphrased → ambiguous queries.

Finding 4: Interface Complexity Reduces Accuracy

All models perform worse on larger interfaces.

Finding 5: Instruction Tuning Matters More Than Model Size

Gemma 2 (2B) performs statistically equal to Llama 3 (8B).

7. Statistical Analysis

McNemar's Test

McNemar's test evaluates whether differences between two classifiers are statistically significant.

Key Result

- Gemma 2 vs Llama 3 → Not Significant
- All other comparisons → Significant

This confirms that several observed performance gaps are meaningful and not caused by randomness.

8. Discussion

8.1 Query Ambiguity Dominates Failure

Ambiguous queries produce larger performance degradation than interface size.

8.2 Small Models Have Capability Limits

Models below approximately 2B parameters struggle to solve this task.

8.3 Instruction-Tuning Outweighs Raw Parameters

Model quality depends more on fine-tuning strategy than model size.

8.4 Compound Degradation Exists

Ambiguity + large interfaces create non-linear difficulty increases.

9. Threats to Validity

Construct Validity

Dataset created manually may introduce author bias.

Internal Validity

Prompt design may influence performance.

External Validity

Only GitHub and AWS interfaces are evaluated.

Reliability

Temperature = 0 ensures reproducibility.

10. Conclusion

This study presents one of the first controlled investigations into how query ambiguity and interface complexity affect LLM-based UI element selection in developer tools.

Results demonstrate that:

- Query ambiguity is the dominant performance challenge.
- Interface complexity amplifies failure.
- Small instruction-tuned models can compete with larger models.
- Mistral 7B performs best overall.
- Models below ~2B parameters are generally insufficient.

The findings support future research in UI agents, developer assistants, and intelligent automation systems.

11. Future Work

Future research may explore:

1. Few-shot prompting
 2. Chain-of-thought prompting
 3. Additional platforms beyond GitHub and AWS
 4. Mobile UI evaluation
 5. Real user-generated queries
 6. Cross-language UI datasets
 7. Fine-tuned UI-specialized models
-

Final Research Positioning Statement

This paper contributes a reproducible benchmark and controlled evaluation framework for studying LLM-based UI element selection under ambiguity and interface complexity conditions in developer environments.

The work positions itself at the intersection of:

- Human–Computer Interaction
- LLM Evaluation
- Intelligent User Interfaces
- Software Engineering Tool Support
- Agentic Interface Navigation